Welcome to the world of AI/ML! Because you are starting completely from scratch but are a quick learner, we are going to treat this project as a step-by-step assembly line. You do not need to read a massive 500-page textbook on AI math before you can build this. Instead, you will learn the absolute necessary components day by day.
Here is your highly detailed, day-by-day learning schedule and implementation timetable for the **"Are You a Cat?" Production MLOps system**, directly mapping out every single detail described by Marina Wyss at [00:07:55].

📅 Week 1: Core AI Foundations & The Local Baseline

Goal: Learn how a computer looks at an image, download your sample data, and write your first neural network code to output a basic model file (.h5).

Day	Task	Learning Notes	Learning Resources
Day 1	**Python Environment & VS Code Setup**	Set up a pristine Python environment. Do not install libraries globally; always isolate your projects using a Virtual Environment (venv).	📖 Python Virtual Environments Guide
Day 2	**Understand how Image Coding Works**	Learn how images are processed as tensors (multi-dimensional pixel arrays with red, green, blue color channels) and how Convolutional Neural Networks (CNNs) pull out shapes and patterns.	🎥 3Blue1Brown: What is a Neural Network? 📖 How CNNs Work Intuitively
Day 3	**Data Ingestion Setup**	Create a local directory structure: data/train/cat/,	🗂️ Kaggle Cats & Dogs Images

		data/train/dog/, and data/train/human/. Download 100 sample images for each class using standard open datasets.	🗂️ LFW Dataset (For human selfies)
Day 4	**Write Your First Training Script**	Write a simple script using TensorFlow/Keras. Use image_dataset_from_directory to automatically load, resize (to 150x150 pixels), and batch your images.	📖 TensorFlow Loading Images Tutorial
Day 5	**Compile & Fit Your Model**	Write a small 2D CNN model with 3 Convolutional layers followed by a Flatten layer and a dense Softmax output layer (3 classes). Run model.fit() for 5 simple epochs.	📖 TensorFlow Image Classification Starter
Day 6	**Save the Model Artifact**	Append model.save('baseline_cat_model.h5') to the end of your script. Run it, verify that it finishes executing, and ensure the .h5 file appears on your local disk.	📖 Keras Saving & Loading Models Guide

Day 7	**Week 1 Review & Git Init**	Run a git init command, create a .gitignore to prevent committing your raw images, and push your week 1 script up to GitHub.	📖 GitHub Skills: Introduction to Git

📅 Week 2: Build the UI Application & Production Feedback Loop

Goal: Build a functional, interactive user interface where users can upload a selfie and submit performance feedback, which you'll save directly to your local file system to simulate cloud storage.

Day	Task	Learning Notes	Learning Resources
Day 8	**Introduction to Streamlit**	Streamlit lets you construct clean web application frontends using 100% Python. Learn basic input/output components like st.title and st.write.	📖 Streamlit Starter Documentation
Day 9	**Build the Image Upload UI**	Add an st.file_uploader() wrapper code layer to allow your system to accept .jpg and .png image strings from your browser.	📖 Streamlit File Uploader API API
Day 10	**Connect UI to the Baseline Model**	Use tf.keras.models.load_model() inside	📖 Preprocessing Images for Keras Predictions

		your app to load your Week 1 model file. Preprocess the uploaded image and run model.predict() to display the output.	
Day 11	**Build the User Feedback UI**	Under the prediction output, add st.button("Correct") and st.button("Incorrect") components so users can click to log whether the system's prediction was right or wrong.	📖 Streamlit Buttons & State Management
Day 12	**Simulate Cloud Storage**	Create a local data directory data/s3_feedback_bucket/. When a user clicks a feedback button, write Python code to save the file with a unique timestamp name (e.g., 20260529_1159_correct.jpg).	📖 Python built-in open() & os file methods
Day 13	**End-to-End UI Validation**	Open your app (streamlit run app.py), upload your own selfie, check the classification result, click "Correct", and verify the file saves	🎥 Streamlit Web Apps in 10 Minutes

		cleanly inside your simulated bucket.	
Day 14	**Week 2 Code Refactoring**	Clean up your application scripts, ensure paths are relative, and commit the functional app architecture to Git.	🛠️ *Self-guided debugging and architecture review.*

📅 Week 3: Automated Data Validation & Experiment Tracking

Goal: Implement automated data validation guardrails using **Deepchecks** and hook up **MLflow** to track metrics, parameters, and version histories across multiple runs.

Day	Task	Learning Notes	Learning Resources
Day 15	**What is Data Validation?**	Understand why bad data (blurry photos, wrong image dimensions, mismatched labels) can ruin production models.	🎥 Introduction to Machine Learning Data Quality
Day 16	**Write Deepchecks Guardrails**	Install Deepchecks (pip install "deepchecks[vision]"). Write a script that checks your data directory for duplicate image configurations or corrupt files before training starts.	📖 Deepchecks Vision Data Validation Guide
Day 17	**Introduction to**	Understand why	📖 MLflow Tracking

	MLflow	using a shared tracking dashboard beats scribbling evaluation metrics down manually in a text document.	Core Concepts
Day 18	**Integrate MLflow Autologging**	Add import mlflow and mlflow.tensorflow.autolog() directly ahead of your Keras training compilation logic. Run a new training sequence.	📖 MLflow + TensorFlow Integration Specs
Day 19	**Explore the Local MLflow UI**	Spin up the server engine locally using your terminal command line (mlflow ui). Open http://127.0.0.1:5000 to browse your run parameters and loss curves.	📽️ MLflow Experiment Tracking Crash Course
Day 20	**Logging Weights & Custom Artifacts**	Modify your training code to explicitly save model metrics (Precision and Recall) and the absolute model artifact structure using mlflow.log_artifact().	📖 MLflow Logging Metrics & Artifacts API
Day 21	**Week 3 Integration Check**	Run two separate training configurations (one	🛠️ *Self-guided data verification.*

		with a learning rate of 0.01 and one with 0.001) and contrast their metrics side-by-side using the MLflow web table views.	

📅 Week 4: Pipeline Orchestration with ZenML

Goal: Stop executing your training, validation, and metadata logging scripts manually. Use **ZenML** to modularize your steps into an automated workflow.

Day	Task	Learning Notes	Learning Resources
Day 22	**Introduction to Pipeline DAGs**	Learn why production code requires Directed Acyclic Graphs (DAGs) to separate concerns into specific, reproducible step chains.	📖 ZenML Starter Guide: ML Pipelines
Day 23	**Install & Boot ZenML Server**	Run pip install "zenml[server]" followed by zenml init and zenml login --local. This spins up a visual interface tracking pipeline histories.	📖 ZenML Official Setup Quickstart
Day 24	**Create Data Ingestion Steps**	Convert your raw data extraction processes into a ZenML step function using the	📖 ZenML Step Definitions Docs

		code descriptor @step.	
Day 25	**Create Deepchecks Step Components**	Move your Week 3 Deepchecks validation suite into a clean @step def data_validation_step() wrapper that outputs data quality checks.	📖 ZenML Advanced Steps & Typing
Day 26	**Define Your First Training Pipeline**	Create a run_pipeline.py script containing your sequential steps inside a unified @pipeline block.	📖 ZenML Pipeline Configurations
Day 27	**Run and Monitor via Pipeline Dashboard**	Execute your pipeline from the terminal (python run_pipeline.py). Log into your local dashboard endpoint (http://127.0.0.1:8237) to view your visual execution graph.	📽️ ZenML Open Source Dashboard Walkthrough
Day 28	**Week 4 Review & Git Backup**	Ensure all component steps execute cleanly without dependency crashes, then update your remote	🛠️ *Self-guided code consolidation.*

		GitHub repository.	

📅 Week 5: Building the Deployment Gatekeeper (The MLOps Core)

Goal: Connect ZenML with your MLflow instance and implement a conditional deploy rule to act as an automated gatekeeper.

Day	Task	Learning Notes	Learning Resources
Day 29	**Understand ZenML Stacks**	Learn how ZenML abstractly links independent orchestration components (orchestrator, experiment tracker, artifact store) together using modular stack profiles.	📖 ZenML Core Components: Stacks
Day 30	**Register the MLflow Tracker Profile**	Execute CLI stack registry commands to connect MLflow with your active ZenML pipeline environment configurations.	zenml experiment-tracker register mlflow_track --flavor=mlflow zenml stack register cse_stack -e mlflow_track --set
Day 31	**Write the Evaluation Metrics Parser**	Create an evaluation step function that accurately extracts precision, recall, and accuracy numbers from test	📖 Evaluating Keras Models programmatically

		data splits.	
Day 32	**Implement the Deployment Trigger Gate**	Write a custom step: @step def deployment_gate(accuracy: float) -> bool. Return True only if accuracy beats your target floor (e.g., 0.80).	🛠️ *Review the code blueprint provided in our previous response.*
Day 33	**Automated Model Serving Configuration**	If the deployment gate evaluates to True, use MLflow's tracking client API to label that specific run version as the official Production model.	📖 MLflow Model Registry Client API Specs
Day 34	**Test a Broken Deployment Configuration**	Intentionally break your pipeline inputs (e.g., pass bad data batches) to verify that the automated gatekeeper successfully halts and prevents deployment.	🛠️ *Chaos testing simulation.*
Day 35	**Week 5 Execution Complete**	Ensure that your pipeline is automatically training models, calculating metrics, and flagging passing versions as Production in MLflow.	🛠️ *Log validation audit.*

📅 Week 6: Closing the Loop & Portfolio Packaging

Goal: Update your user interface to fetch models dynamically from MLflow, test the complete data loop, and document the project on GitHub to impress tech recruiters.

Day	Task	Learning Notes	Learning Resources
Day 36	**Decouple the Web UI from File Paths**	Open your frontend.py script. Remove the hardcoded file paths (e.g., baseline_cat_model .h5).	🛠️ *Refactor frontend architecture.*
Day 37	**Fetch Dynamic Production Models**	Integrate the MLflow Python API inside your Streamlit app to look up and load whichever model version holds the active Production tag.	📖 Fetching Models from MLflow Registry via Code
Day 38	**Verify the Dynamic Update Loop**	Run your training pipeline to produce a better model variant. Refresh your open Streamlit tab and confirm it automatically loads the updated model without rebooting the server.	🎥 Connecting Streamlit UI to Live API Models
Day 39	**Data Drift Concepts**	Learn why saving real user photos into your local data lake folder sets up future systems for data drift checks	📖 What is Data Drift & Concept Drift?

		and continuous training.	
Day 40	**Refactor Code & Final Polish**	Remove dead log prints, structure your codebase directories cleanly, and verify that requirements are properly documented in a requirements.txt file.	📖 Pip Requirements Files Guide
Day 41	**Write an Elite Repository README**	Write a structural overview containing an architecture chart, system dependencies, a listing of your tech stack choices, and concrete execution instructions.	📖 How to Write a Professional Project README
Day 42	**Record Demo & Share to Portfolio**	Record a 2-minute screen recording showcasing your pipeline passing a data execution check, updating MLflow, and serving the model live to your Streamlit app interface.	🚀 *Share your creation with recruiters and peers on LinkedIn!*

💡 Crucial Notes for Success:

- **Don't skip days:** Dedicating just **1 hour every single day** will help you build muscle memory far faster than trying to cram 7 hours of coding into a single weekend.
- **Launch your servers first:** Remember that when running your codebase in weeks 4, 5, and 6, you need to open separate terminal windows to keep your zenml server and

mlflow ui active in the background.